Springboot: Dependency Injection

What is Dependency Injection

- Using Dependency Injection, we can makes our class independent of its dependencies.
- It helps to remove the dependency on concrete implementation and inject the dependencies from external source.

Let's see the Problem first:

```java
public class User {

    Order order = new Order();

    public User(){
        System.out.println("initializing user");
    }
}
```

```java
public class Order {

    public Order(){
        System.out.println("initializing Order");
    }
}
```

Issues with above class structure:

1. Both User and Order class are **Tightly coupled**.
   - Suppose, Order object creation logic gets changed *(lets say in future Object becomes an Interface and it has many concrete class)*, then USER class has to be changed too.

```java
public class User {

    Order order = new Order();

    public User(){
        System.out.println("initializing user");
    }
}
```

new OnlineOrder();

```java
public interface Order {
}
```

```java
public class OnlineOrder implements Order{
}
```

```java
public class OfflineOrder implements Order{
}
```

2. It breaks **Dependency Inversion** rule of S.O.L.I.D principle
   i. This principle says that DO NOT depend on concrete implementation, rather depends on abstraction.

Breaks Dependency Inversion Principle (DIP)

```java
public class User {

    Order order = new OnlineOrder();

    public User(){
        System.out.println("initializing user");
    }
}
```

Follows Dependency Inversion Principle (DIP)

```java
public class User {

    Order order;

    public User(Order orderObj) {
        this.order = orderObj;
    }
}
```

Now in Spring boot how to achieve Dependency Inversion Principle?

Through Dependency Injection

- Using Dependency Injection, we can makes our class independent of its dependencies.
- It helps to remove the dependency on concrete implementation and inject the dependencies from external source.

```java
@Component
public class User {

    @Autowired
    Order order;

}
```

```java
@Component
public class Order {
}
```

@Autowired, first look for a bean of the required type.
-> If bean found, Spring will inject it.

Configuration